How push_swap works, end to end

From the command line to the printed operations: main.c, input/, benchmark/, operations/

The job: take a list of integers (stack A, first argument on top) and print the shortest list of operations that sorts it, using a second stack B and only the 11 allowed moves. This walkthrough covers everything around the sort itself; the three strategies have their own PDFs (simple, medium, complex).

1 Join

All of argv becomes one string, then is split on spaces. Quoted groups and separate words end up the same.

`build_args`

2 Validate

Every word must be a valid int with no duplicates, or a known flag (at most one strategy, one --bench).

`check_args`

3 Parse

Numbers become linked-list nodes in A, flags are recorded, then every node gets its rank.

`parse_args / assign_index`

4 Sort

Measure the disorder, then run the strategy from the flag, or pick one from the disorder.

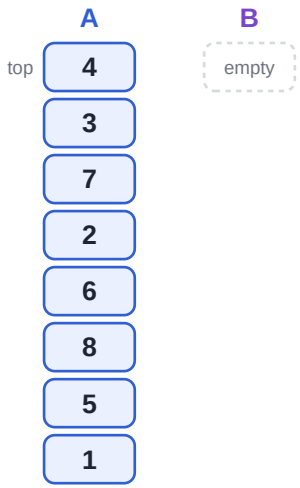
`init_bench / sort_stacks`

5 Output

Each operation prints itself as it runs. With --bench, a summary goes to stderr.

`print_bench`

THE EXAMPLE IN THIS WALKTHROUGH



Input `./push_swap --bench 4 3 7 2 6 8 5 1`: the same numbers as the algorithm walkthroughs, plus `--bench` so every stage of the program has something to show.

No strategy flag is given, so the program runs in adaptive mode: it measures how unsorted the input is and picks the complex sort. It prints **19 operations** to stdout and a 5-line report to stderr.

`main` is a straight line: each stage either succeeds or ends the program with `Error` on stderr and exit code 1. With no arguments at all (`argc == 1`) it returns 0 without printing anything.

All tables of inputs and outputs in this PDF were produced by running the real binary.

The data the program carries around

Four structs from push_swap.h. Stacks are singly linked lists; the head is the top.

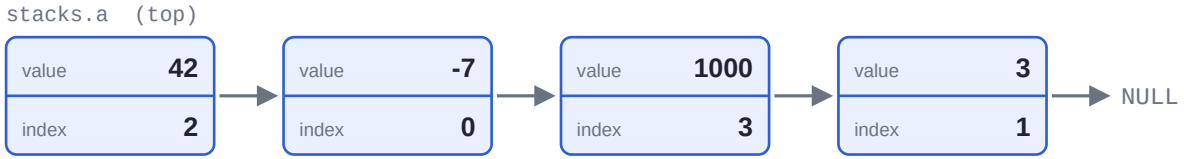
```
typedef struct s_list {
    int            value;  // the number
    int            index;  // its rank, 0..n-1
    struct s_list  *next;  // node below
}   t_list;

typedef struct s_stacks {
    t_list        *a;
    t_list        *b;
    t_operations  operations; // 11 counters
}   t_stacks;                 // + total

typedef struct s_flags {
    t_mode  mode;   // DEFAULT, SIMPLE, ...
    int     bench;  // 1 if --bench
}   t_flags;

typedef struct s_bench {
    float  disorder;
    char   *strategy;
    char   *complexity;
}   t_bench;
```

Stack A for ./push_swap 42 -7 1000 3



`stacks.b = NULL (B starts empty)`

Top is the head. push, swap and rotate only relink the first one or two pointers, so pa, pb, sa and sb are O(1). ra and rra have to walk to the last node, so they are O(n) in this list (it has no tail pointer).

value vs index. `value` is what the user typed. `index` is its rank among all inputs: 0 for the smallest, n-1 for the largest. The sorts and the disorder measure only ever compare ranks, so they do not care about negatives or huge numbers.

Counting. Every operation function bumps its own counter and `total` in `stacks.operations`, whether or not it prints. That is what `--bench` reports at the end.

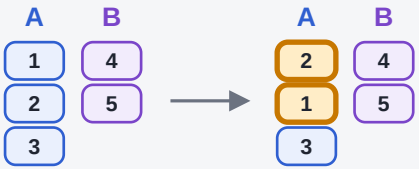
Modes. `t_mode` is DEFAULT, SIMPLE, MEDIUM, COMPLEX, ADAPTIVE or BENCH. `get_mode` maps a word to one of them; anything that is not one of the five flags is DEFAULT, which means "should be a number".

The 11 operations

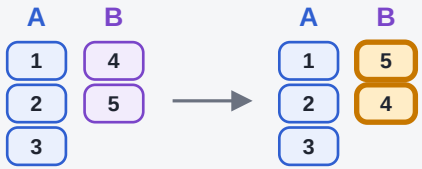
moved by the operation

operations/: swap.c, push.c, rotate.c, reverse_rotate.c. Every cell starts from the same stacks.

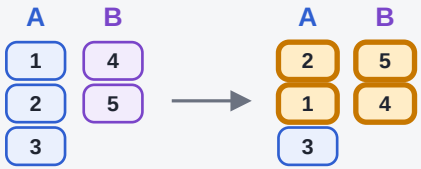
`sa` swap the top two of A



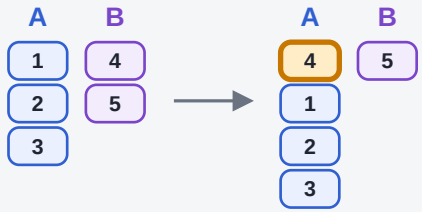
`sb` swap the top two of B



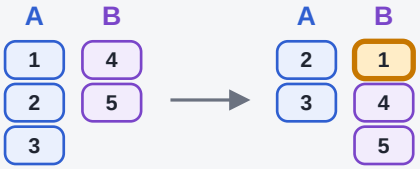
`ss` sa and sb at once



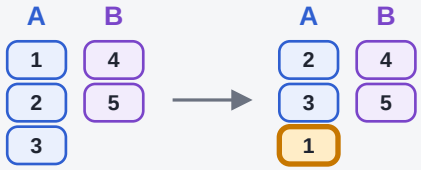
`pa` move the top of B onto A



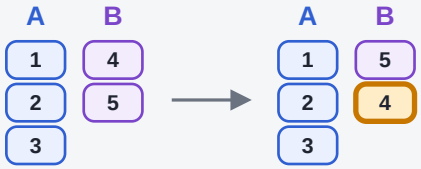
`pb` move the top of A onto B



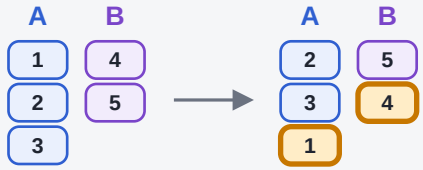
`ra` top of A goes to the bottom



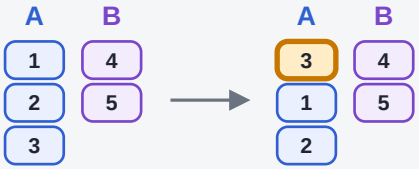
`rb` top of B goes to the bottom



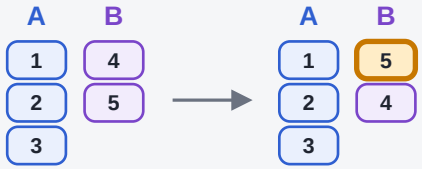
`rr` ra and rb at once



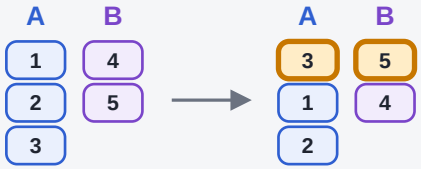
`rra` bottom of A comes to the top



`rrb` bottom of B comes to the top



`rrr` rra and rrb at once



Shared rules

With fewer than 2 numbers (or an empty source for pa/pb) the move changes nothing, but it is still printed and counted. Each function takes `print`: push_swap passes 1, the checker passes 0.

Step 1: join everything, then split on spaces

input/parse.c: build_args turns argv into one NULL-terminated array of words.

How it works

`build_args` starts from `ft_strdup("")` and appends every `argv[i]` from 1 on, with one space between them. Each append goes through `str_join_free`, which frees the old string so nothing leaks.

Then `ft_split` cuts the string at spaces. Runs of spaces count as one separator and leading or trailing spaces disappear, so the words come out clean.

From here on the program only looks at `args`. It is freed at the end of `main`, or by `print_error` on the way out.

What that means for the user

command line	words	output
`"3 2 1"`	3	`ra sa`
`"3 2" 1`	3	`ra sa`
`3 2 1`	3	`ra sa`
`"  3   2 1  "`	3	`ra sa`
`"--complex 3 1 2"`	4	`ra`
`2 1 ""`	2	`sa`
`""`	0	`(nothing)`

Quoting makes no difference, flags can sit inside a quoted group, and empty strings vanish. Only the space character separates words: a tab stays inside the word and fails validation later.

Step 2: validate every word before touching the stacks

input/check.c: check_args walks args once; the first problem prints Error and exits.

The checks, in order

1. Flag or number? `get_mode` compares the word with the five flags. A match is a flag; anything else must be a number.

2. Number format (`check_number`): optional + or -, then at least one digit, and only digits. No spaces, letters or decimal points.

3. Range: `ft_atol` converts it to a long, which must be between -2147483648 and 2147483647.

4. Duplicates (`check_duplicate`): compared by value with every earlier number, so 0 and -0, or 5 and +5, are duplicates.

5. Flags (`check_exist`): one strategy flag (--simple, --medium, --complex, --adaptive) and one --bench at most. A second one of either kind is an error.

On error `print_error` frees `args`, writes `Error` to stderr and calls `exit(1)`. Nothing has been printed to stdout yet, and the stacks do not exist yet.

command line	output	why
`3 2 1`	`ra sa`	all valid
`+5 3`	`sa`	a leading + is fine
`2147483647 -2147483648`	`sa`	the int limits are allowed
`2147483648`	`Error (exit 1)`	above INT_MAX
`1 a`	`Error (exit 1)`	not a number
`- 1`	`Error (exit 1)`	a sign with no digits
`"1\t2"`	`Error (exit 1)`	a tab is not a separator
`0 -0`	`Error (exit 1)`	duplicate: both are 0
`--simple --medium 2 1`	`Error (exit 1)`	two strategy flags
`--bench --bench 2 1`	`Error (exit 1)`	two --bench
`--fast 2 1`	`Error (exit 1)`	unknown flag: not a number
`2 1 --simple`	`sa`	flags can go anywhere

Step 3: build stack A and rank every number

input/parse.c and input/index.c: parse_args, then assign_index.

parse_args

A second pass over `args` (they are known to be valid now). Each number becomes a node from `ft_lstnew` with `index = -1`, appended with `ft_lstadd_back`, so the first argument ends up on top.

Flags are recorded instead: `--bench` sets `flags.bench = 1`, a strategy flag sets `flags.mode`. Without one, the mode stays DEFAULT, which sort_stacks treats as adaptive.

Finally `assign_index(stacks->a)` gives every node its rank.

assign_index

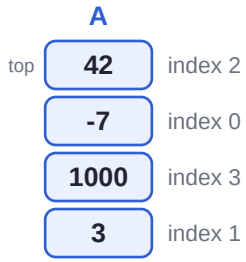
Repeat: `get_next_min_node` scans the whole list for the smallest value whose index is still -1, and it gets the next rank (0, 1, 2, ...). When every node has a rank the scan returns NULL and the loop ends.

n rounds of an n-node scan: O(n^2), which is fine at 500 numbers. Duplicates were already rejected, so every rank is unique.

EXAMPLE: ./push_swap 42 -7 1000 3

round	smallest unranked	gets index	A, as value (index)
1	-7	0	42 (-1) -7 (0) 1000 (-1) 3 (-1)
2	3	1	42 (-1) -7 (0) 1000 (-1) 3 (1)
3	42	2	42 (2) -7 (0) 1000 (-1) 3 (1)
4	1000	3	42 (2) -7 (0) 1000 (3) 3 (1)
5	none left		loop ends

RESULT



From now on the sorts compare these indexes. For the example in this walkthrough, 4 3 7 2 6 8 5 1, the index is simply value minus 1.

Step 4: pick a strategy and sort

benchmark/init.c measures the disorder first; algorithms/sort.c then chooses what to run.

The flag decides

flag	runs	label
`--simple`	`insertion_sort`	O(n^2)
`--medium`	`chunk_sort`	O(n sqrt n)
`--complex`	`quick_sort`	O(n log n)
`--adaptive or none`	`adaptive_sort`	by disorder

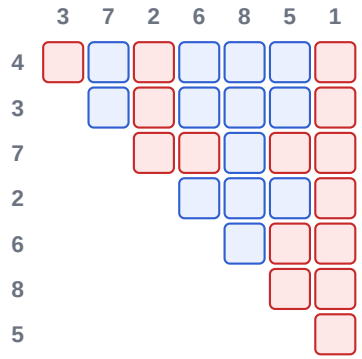
Every strategy returns at once if A is already sorted, and hands 5 numbers or fewer to fixed_sort. The three strategies are explained in their own PDFs.

`sort_stacks` is called with `print = 1`, so each operation is written to stdout the moment it runs.

When is it computed? `init_bench` always runs, right after parsing and before any operation, on the ranks in A. With a strategy flag the number is only used for the --bench report.

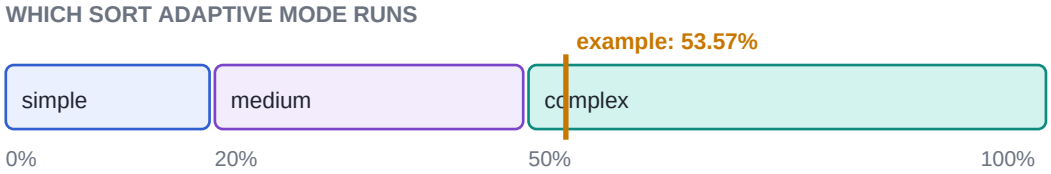
Adaptive mode: measure the disorder

`compute_disorder` looks at every pair (i before j) and counts the ones in the wrong order (index i > index j). Disorder = wrong pairs / all pairs: 0 for sorted, 1 for reversed.



Row = earlier number, column = later number. A red cell is a pair in the wrong order.

For 4 3 7 2 6 8 5 1: **15 of 28** pairs are wrong, so the disorder is 15/28 = **53.57%**.



Below 20% the input is nearly sorted and insertion is cheap; from 50% up it is mostly shuffled. The example lands just past 50%, so adaptive mode runs the complex sort and sets the label to O(n log n).

Step 5: what comes out

stdout gets the operations as they happen; stderr gets the --bench report at the end.

STDOUT: 19 LINES, ONE OPERATION EACH

`pb` `pb` `rb` `ra` `pb` `rb` `rra` `rra` `pb` `rra` `pb` `sa` `pa` `pa` `ra` `pa` `pa` `pa` `rra`

STDERR: THE --BENCH REPORT (print_bench, fd 2)

```
[bench] disorder:  53.57%
[bench] strategy:  Adaptive / O(n log n)
[bench] total_ops:  19
[bench] sa:  1  sb:  0  ss:  0  pa:  5  pb:  5
[bench] ra:  2  rb:  2  rr:  0  rra:  4  rrb:  0  rrr:  0
```

Cleaning up

After sorting (and the report, if asked for), `main` frees the word array with `free_args` and every node in both stacks with `free_stacks`, then returns 0. On an error path, `print_error` frees the words before exiting; the stacks are only built after validation, so there is nothing else to free.

Two streams. Operations go to fd 1 and the report to fd 2, so piping the output into the checker still works with --bench on.

disorder is printed with two decimals, cut off rather than rounded (`print_disorder` splits the percentage into its integer part and the first two decimals).

strategy says which mode ran and its complexity label. In adaptive mode it reads Adaptive, and the label is the one of the strategy that was picked.

counts come from the counters every operation increments, so they always add up to total_ops.

Quick reference

Where everything lives, and how to build and run it.

Files

folder	files	role
`main.c`	main	the five stages, in order
`input/`	parse, check, flags, stacks, index	join, validate, parse, rank, free
`benchmark/`	init, print	disorder, labels, the --bench report
`algorithms/`	sort, simple*, medium, complex, ...	strategy choice and the sorts (own PDFs)
`operations/`	swap, push, rotate, reverse_rotate	the 11 moves and their counters
`libft/`	ft_split, ft_atol, ft_lst*, ft_put*, ...	string, number, list and output helpers
`bonus/`	main, input, operations	the checker (own PDF)

Build

target	does
`make`	builds push_swap
`make bonus`	builds checker
`make clean`	removes the .o files
`make fclean`	also removes both binaries
`make re`	fclean, then all

Run

```
./push_swap 4 3 7 2 6 8 5 1
./push_swap --simple 4 3 7 2 6 8 5 1
./push_swap --bench 4 3 7 2 6 8 5 1

ARG="4 3 7 2 6 8 5 1"
./push_swap $ARG | ./checker $ARG
./push_swap $ARG | wc -l
```

Exit code 0 on success, 1 with `Error` on stderr for invalid input.

With no arguments the program prints nothing and exits 0.

Strategy flags: --simple, --medium, --complex, --adaptive (the default). Add --bench for the report.